Software Architecture Specifications

Version: 1.0
Last Updated: 03 September 2026

momently

Prepared by

CYBERNAUTS x momentum

momently
Timesheets AI

Introduction

This document outlines the architectural foundation of the Momently system. The architecture described in this document serves as the blueprint for the system's design, implementation, and evolution. It establishes the structural framework that guides the development of the system, ensuring that all architectural decisions remain aligned with the project's functional and non-functional requirements.

As the project progresses, this document acts as a reference to help ensure that development remains consistent with the intended architecture. It explains the reasoning behind the technologies, architectural patterns, design patterns, and deployment strategies that have been selected, making it easier to maintain consistency as new features are added or existing functionality is refined. By documenting these decisions, the architecture can evolve in a structured way while continuing to support important quality attributes such as scalability, performance, security, and maintainability.

Architectural Requirements

Architectural Requirements

Security and Access Control

- Users must securely authenticate using either their email and password, Single Sign On (Google and Microsoft) and admin accounts should have Multifactor Authentication mechanisms
- Role-Based Access Control (RBAC) must restrict actions based on workspace roles of Admin, Manager or Developer
- All passwords must be encrypted
- Every significant action must be audited with timestamp, user ID, and IP address

Multi-Tenancy

- The system must support multiple workspaces using the same application instance

- Data must be strictly isolated between workspaces
- Users must be able to belong to multiple workspaces

Performance and Scalability

- The system must remain responsive during peak usage (examples include early in the morning at 8am when users start work)
- Dashboard and reporting queries must load within acceptable timeframes
- The system should support horizontal scaling as user count grows

Reliability and Availability

- The system must handle partial failures gracefully
- Critical operations must not lose data (examples include timesheet submissions or approvals)
- External integrations (Jira, GitHub, Calendar) should not block core functionality if they fail

Modifiability and Extensibility

- New integration types should be easy to add

Data Integrity and Consistency

- Time entries must maintain referential integrity
- Timesheet approvals must prevent modifications to approved entries

Regulatory Compliance

- Detailed audit trails for critical operations
- Soft deletion to preserve historical data
- Ability to export data for compliance reporting

Architectural Patterns

Overview of Subsystems

Momently is composed of six primary subsystems, each responsible for a distinct domain:

- **Time Tracking subsystem:** timer sessions, manual time entries, duration calculation
- **Timesheet & Approval subsystem**: timesheet generation, submission, approval or rejection
- **Reporting & Analytics subsystem**: dashboards, productivity metrics, AI-powered insights and forcasting

- **Integration subsystem**: synchronisation with Jira, GitHub and Google Calender
- **Identity & Access subsystem**: user authentication, MFA, SSO, RBAC and workspace membership
- **Leave Management subsystem**: leave requests, approvals, availability tracking

4-Layer Architecture (Primary Pattern)

The system utilises the layered architecture pattern because it enforces separation of concern as each layer has a well-defined responsibility. Using a 4-layered architecture organizes core application logic into horizontal, decoupled layers. This strict separation of concerns improves maintainability, testability, and scalability.

This architectural pattern supports our requirements in the following ways:

1. Each layer can be changed independently as long as the interfaces remain stable
2. The persistence layer will handle transactional consistency
3. The services layer can be scaled horizontally

The secondary pattern is a Client-Server pattern where the client is the Angular frontend running in the browser, and the server is the SpringBoot backend exposing the REST APIs. This allows a clear separation between the User Interface and the business logic. Furthermore, the server components have independent deployment and scaling capabilities.

MVVM (Model-View-ViewModel) - Frontend Pattern

The frontend follows the MVVM (Model-View-ViewModel) architectural pattern, which separates concerns into three distinct layers:

- Model: Represents raw application data
- View: The user interface
- ViewModel: Acts as an intermediary that manages application state and logic

The ViewModel transforms raw data into a format suitable for presentation and handles communication between the View and backend services. This ensures a clear separation between UI logic and business/data logic, improving maintainability and testability.

Layer Descriptions

Following the standard SpringBoot Architecture (GeekForGeeks, 2026), the system is divided into four layers, each responsible for a specific aspect of the application processing:

Presentation Layer

The Presentation Layer encompasses both the client-side user interface (using the MVVM pattern described above) and the server-side API endpoints. The Spring Bot controllers act as the entry point for all incoming HTTP requests by exposing the RESTful APIs consumed by the frontend. Further, they forward the validated requests to the services layer.

Services Layer

The Services Layer contains the core business logic of the system. It handles operations such as timesheet processing, approvals, reporting, and external integrations.

This layer also incorporates supporting services such as:

- Redis for caching and performance optimization
- FastAPI-based AI services for advanced analytics (e.g. productivity prediction and forecasting)
- Integration adapters for external tools such as Jira, Google Calendar, and GitHub

Persistence Layer

The Persistence Layer manages data access and transformation. It uses Spring Data JPA/Hibernate to map application objects to relational database structures through repository abstractions.

Database Layer

The Database Layer represents the physical storage infrastructure where the application data persists. The technology used for this layer is the relational database PostgreSQL.

momently
Timesheets AI

Architectural Diagram

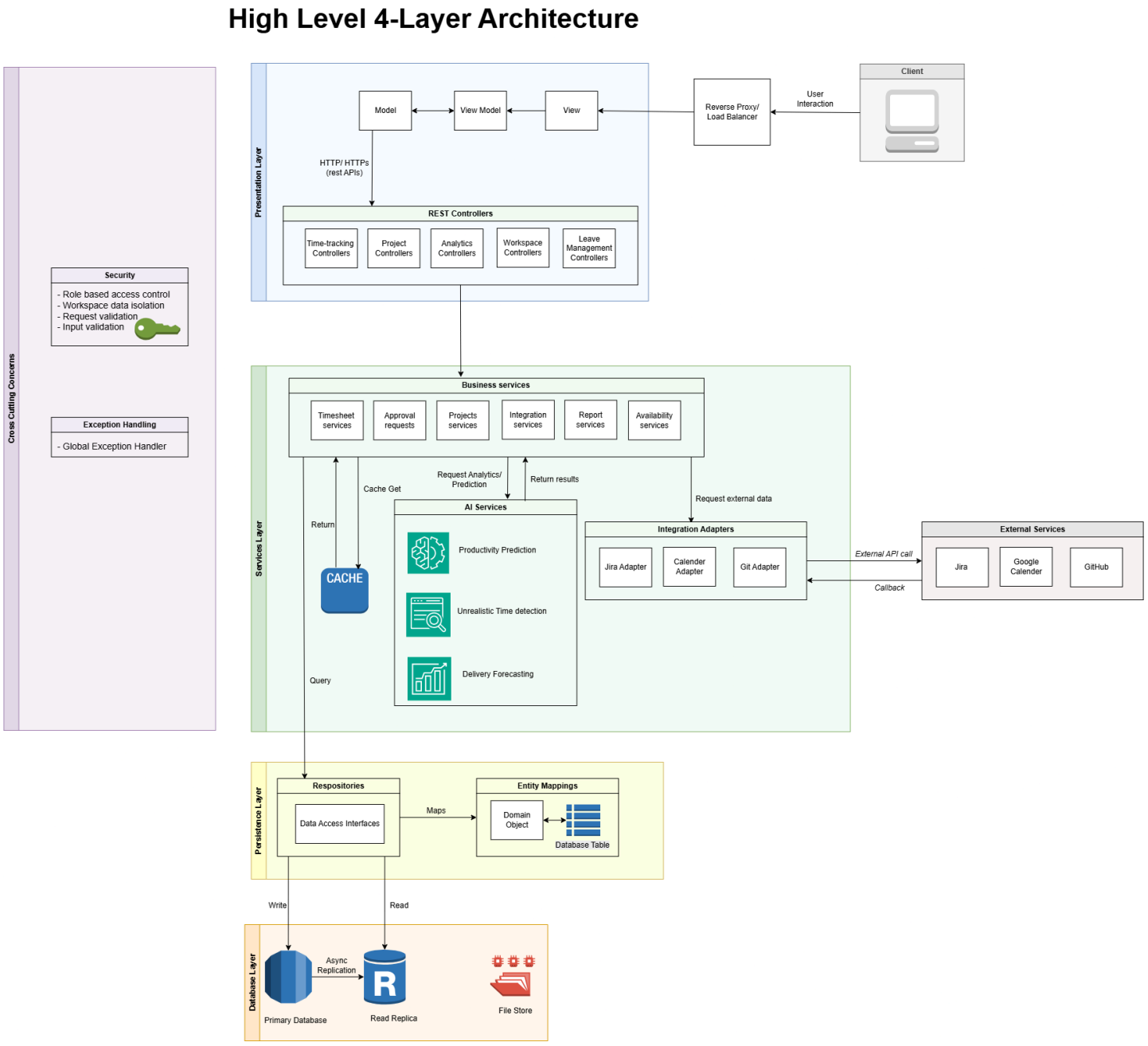


Figure 1: Architectural diagram

momently
Timesheets AI

NFR Traceability Matrix

ID	Quantified requirement	Tactic in SAS	Test / tool	Target/Actual
QR-01	Performance: p95 of request at 100 concurrent users	- Redis caching - Database indexing -Asynchronous processing - Optimized JPA queries using JOINS - Stateless services layer	-Apache JMeter -Spring Boot Actuator -Prometheus -AWS CloudWatch	=< 2.0s (Error rate < 1%)/
QR-02	Scalability: 200% workload increase with ≤ 10% degradation	- Stateless services layer - Database read replicas	Apache JMeter Prometheus AWS CloudWatch	=< 10% degradation/
QR-03	Security: 100% authorization required	- Spring Security with JWT - RBAC enforcement - AWS Parameter Store - BCrypt password hashing	-Spring Security -JWT for stateless authentication -AWS Parameter Store -OWASP ZAP	100% authorized access/
QR-03	Security: MFA required for elevated privileges	- MFA enforcement for admin accounts		
QR-04	Availability over the 30 ≥ 99.9% uptime	- Spring Boot Actuator health endpoints - Auto-restart on failure - AWS Application Load Balancer health checks	AWS CloudWatch, UptimeRobot	≥ 99.9% uptime/
QR-04	Availability: Minimum 2hr recovery	- Automated backups -Automated Failover replication	-AWS Backup, PostgreSQL Multi-AZ	Downtime =< 2 hours /
QR-04	Availability: Minimum 1% data loss	- ACID transactions - Foreign key constraints	JUnit integration tests	Data loss < 0.01%) /

			PostgreSQL transaction logs AWS Backup	
QR-05	Maintainability: 80% test coverage	- JUnit 5 and Mockito for unit tests - Jest for frontend tests - JaCoco for coverage reporting - Automated CI/CD execution	JUnit 5, Mockito, Jest, JaCoCo	≥ 80% coverage /
QR-05	Maintainability: Isolated adapter changes	- Layered architecture - Adapter pattern for third party integrations - ArchUnit for architectural enforcement	ArchUnit, SonarQube	Zero architectural violation warnings/

Mapping Quality Requirements to Architectural Decisions

Quality Requirement	Architectural Decision(s)	Trade-off
Performance: 95% of request ≤ 2 seconds	- Redis caching for frequently accessed data - Database indexing - Asynchronous processing for non critical operations - Optimized JPA queries using JOINS	- Caching introduces that stale data may be served - Indexing may slow down write operations - Users may not get immediate confirmation for non-critical operations
Performance: 100 concurrent users	Stateless services layer	Session data must be stored externally
Scalability: 200% workload increase with ≤ 10% degradation	- Stateless services layer - Database read replicas	-Replication lag causing inconsistent reads - Writes still go to primary, which can still cause a bottleneck
Security: 100% authorization required	- Spring Security with JWT - RBAC enforcement - AWS Parameter Store - BCrypt password encoding	- Security checks cause latency on requests - Extra network calls to retrieve secrets
Security: MFA enforcement	- MFA for admin accounts	- MFA can cause user frustration - Increases log in time
Availability ≥ 99.9% uptime	- Health monitoring (AWS cloud watch and Spring Boot Actuator health endpoints) - Auto restart on failure - Load balancer with health checks	- Health checks add overhead - Auto restart may mask underlying problems
Availability: Minimum 2hr recovery	- Automated backups - Failover replication	- Transactions still being processed may be lost (in-flight transactions)
Availability: Minimum 1% data loss	- ACID transactions - Foreign key constraints	- ACID causes database contention and locks - Foreign keys make writes slower
Maintainability: 80% test coverage	- JUnit 5 and Mockito for unit tests - Jest for frontend tests	- Testing add to CI pipeline build time - Time to write tests

	- JaCoco for coverage reporting	- Tests need to be updated as code changes
Maintainability: Isolated adapter changes	- Layered architecture - Adapter pattern for integrations - ArchUnit for architectural enforcement	- Abstraction increases code complexity - Adapter classes increase the number of components to maintain - ArchUnit creates build time overhead

**ACID - Atomacity, Consistency, Isolation, Durability*

Table 1: Mapping Quality Requirements to Architectural Decisions

Design Patterns

Strategy Pattern

The Strategy pattern is used to manage external integrations such as Jira, GitHub, and Google Calendar. Each has different APIs but they serve similar purposes. Each integration strategy defines shared operations, allowing the integration service to swap implementations without changing business logic. Adding a new integration simply requires implementing a new strategy class. This ensures that the system remains unaffected by changes to external services. For example if Jira changes its API only the JiraStrategy needs to be updated.

Question answered: Which integration should I use?

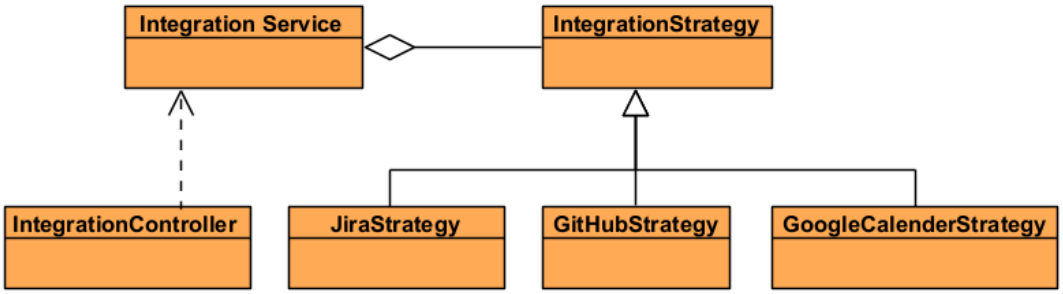


Figure 2: Strategy Pattern
(Context- Integration service, Strategy- IntegrationStrategy, ConcreteStrategies- Jira, GitHub, GoogleCalender Strategies, Client- IntegrationController)

Observer Pattern

When a timesheet gets submitted, several things should happen: notify the manager, update analytics, trigger AI anomaly detection and even send a notification. An event publisher could notify all interested listeners without the timesheet service knowing about each one individually. This makes the system more maintainable and extendable. New behaviors can be added by simply creating new listeners.

Question answered: What happens after an event occurs?

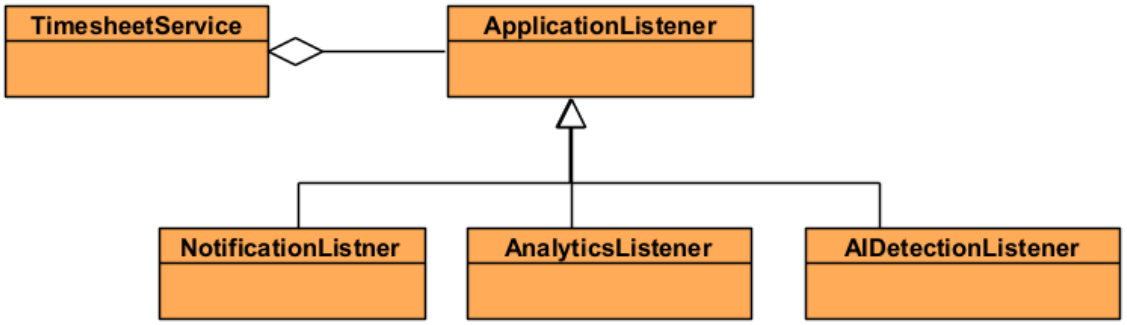


Figure 3: Observer Pattern
(Subject- Timesheet service, Observer- Application Listener, ConcreteObservers- Notification, Analytics, AIDetection listeners)

Proxy Pattern

This pattern will be implemented using Spring's native annotation to create a caching proxy. It acts as the middleman that checks Redis for cached data before calling the real repository, keeping the business logic completely unaware of the caching mechanism.

Question answered: Should I retrieve data from Redis or the database?

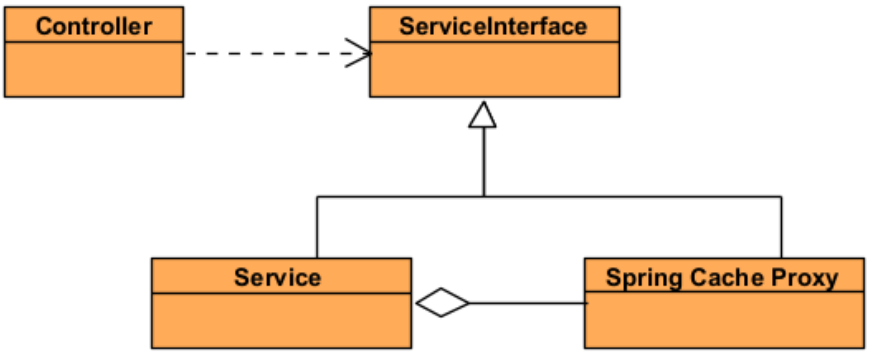


Figure 4: Proxy Pattern

(Client-Controller, Subject- ServiceInterface, Proxy-Spring Cache Proxy, RealSubject- Service Classes)

Class Adapter Pattern

External APIs change, and each integration has its own data formats and API requirements. The Adapter pattern acts as a wrapper that translates between the system's internal data format and the specific API of an external service. For example if Jira changes its API, only the JiraAdapter needs to be updated while the rest of the system remains unchanged. The same approach is used for other integrations such as GitHub and Google Calendar, ensuring the system remains stable, maintainable, and easy to extend with new external tools.

Question answered: How do I communicate with the integration?

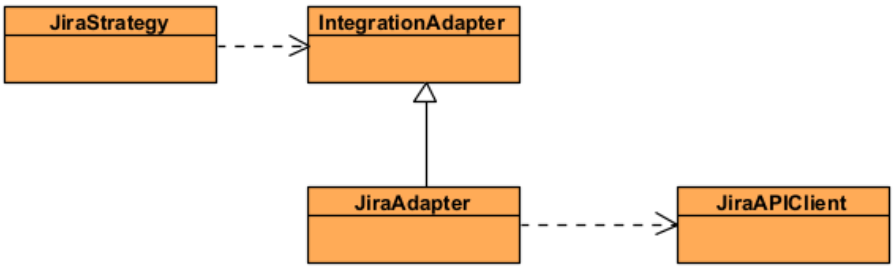


Figure 5: Adapter Pattern
(Client-JiraStrategy, Target- IntegrationAdapter, Adapter- JiraAdapter, Adaptee- JiraAPIClient)

Caching Strategy

Caching plays a critical role in improving system performance and reducing database load. The system checks the cache before querying the database.

- On a cache hit, data is returned directly from Redis.
- On a cache miss, data is retrieved from the database, stored in the cache, and then returned to the client.

This proxy pattern approach reduces database pressure, improves response times, and enhances overall system scalability. Frequently cached data examples:

- User session data
- Dashboard and summary metrics
- External API responses (e.g.Jira, Google Calendar, GitHub)
- Frequently accessed reference data (e.g. active projects, workspace users)
- Productivity scores (e.g. weekly summaries)

Technology Requirements

Frontend Framework

The frontend handles time tracking, analytics, and AI-driven insights, so it needs to stay responsive and accessible. Real-time updates and a modular component architecture are essential because users expect to see their active timer update instantly. Furthermore, the interface should feel efficient even when displaying complex reports or AI-generated recommendations.

Technologies Considered

Angular 17 (Selected)
A TypeScript-based frontend framework developed by Google that provides built-in support for dependency injection, routing, and reactive form handling.

Pros:

- The component-based structure keeps the code organised and easier to maintain, with clear separation of concerns
- Built-in features like routing, form handling, and change detection make it well-suited for a complex user interface like Momently
- TypeScript helps catch errors early, improving overall code quality and maintainability
- Angular Material provides a ready-made, professional UI component library that speeds up development
- It's widely used in enterprise environments, which shows its reliability and long-term stability

Cons:

- Has a steeper learning curve than React
- Requires more setup code for simple components, which can slow down quick development

React
A JavaScript library built around a component-driven approach, supported by a large and active ecosystem.

Pros:

- Has a huge community with a wide range of libraries and tools available
- Very flexible and lightweight to work with
- JSX helps make UI structure easier to read and understand

Cons:

- It's less opinionated, so the team has to make more architectural decisions
- Routing and state management usually require extra libraries
- The flexibility can sometimes lead to inconsistent coding patterns across a team

Vue.js

A progressive framework that is easy to learn and can be introduced gradually into projects.

Pros:

- Easy to pick up, especially for new developers
- Well-documented and beginner-friendly
- Flexible and easy to integrate into existing projects

Cons:

- Less commonly used in large enterprise systems compared to Angular or React
- Not as naturally structured for large-scale applications without additional conventions
- Smaller talent pool in enterprise environments
- Final Selection: Angular 17 + Angular Material

Final selection: Angular 17+

Angular was chosen because its structured, component-based architecture aligns well with the complexity of the system. Features like time tracking, analytics, and multiple external integrations benefit from a clear separation of concerns. Its built-in routing, form handling, and change detection also help maintain fast and responsive interactions, supporting the target 1–2 second response time without needing extra libraries.

Angular enforces consistent development patterns and structure, which helps reduce inconsistency and improves maintainability. Angular Material further supports this by providing a clean, ready-to-use component library that speeds up development. Overall, Angular's strong TypeScript integration, built-in testing support, and enterprise-focused design make it a reliable choice for a large-scale, business-critical application.

Backend Framework and Service Layer

The backend handles authentication, time tracking, project management, timesheet approvals, and external integrations. Security, modularity, and performance are top priorities.

Technologies Considered

Spring Boot 3.5+ with Java 17 (Selected)

Spring Boot is a production-ready framework used to build stand-alone Java applications with embedded servers and minimal configuration. It is widely used for enterprise systems because it provides a structured and scalable development approach.

Pros:

- Built-in support for developing REST APIs and integration testing
- Works naturally with hexagonal and layered architectures
- Java 17 LTS provides long-term stability, security updates, and future compatibility
- Strong encapsulation improves security by reducing unauthorized access risks
- Comprehensive ecosystem including Spring Security, Spring Data JPA, and caching support
- Reliable performance and scalability for enterprise-level applications

Cons:

- Has a steeper learning curve compared to frameworks like Express.js
- Slower startup time during development
- Some manual configuration is still required despite auto-configuration features

Express.js and Node.js

Express.js is a lightweight and flexible framework used for building REST APIs with JavaScript or TypeScript.

Pros:

- Quick and simple to set up
- Large npm ecosystem with many available libraries
- Allows the frontend and backend to use the same language

Cons:

- Provides fewer built-in architectural patterns for large-scale systems

- More difficult to manage complex business logic as the system grows
- Requires more manual handling for security, validation, and transaction management

Final Selection: Spring Boot 3.5+ with Java 17

After evaluating the different backend options, Spring Boot with Java 17 was selected because it provides the best balance of scalability, security, maintainability, and long-term support for a business-critical application.

One of the main advantages is that Spring Boot already includes strong support for REST API development, integration testing, and structured application design, which aligns well with the hexagonal and layered architecture being used in the system.

Java 17 LTS also offers long-term stability, regular security updates, and future compatibility, making it a reliable choice for enterprise development. The framework's strong encapsulation and mature security features help reduce the risk of unauthorized access and support the requirement for secure RESTful APIs. Compared to alternatives such as Express.js, which can become difficult to structure in large systems, or Django, which introduces a separate technology stack, Spring Boot provides a more cohesive solution for the team's skills and project requirements.

Database Management System

The system needs a database that can reliably store and manage time entries, project data, user information, and audit logs. Because this data is highly structured and consistency is critical, the database must support strong transactional guarantees and strict access control.

Technologies Considered

PostgreSQL (Client-required)
PostgreSQL is an advanced open-source relational database that fully supports ACID transactions, strong consistency, and complex querying.

Pros:

- Ensures strong consistency and referential integrity through foreign keys and transactions
- JSONB support allows flexibility when data structures evolve over time

- Includes strong security features such as row-level security and role-based access control, which is useful for multi-tenant or permission-heavy systems
- Already required by the client, making it the default choice

Cons:
- Schema changes require careful planning and proper migration strategies
- Not as naturally suited as NoSQL databases for highly dynamic or schema-less data

MySQL
MySQL is a widely used relational database known for performance and simplicity.

Pros:
- Very fast read performance in many workloads
- Large community and strong industry adoption
- Proven stability in enterprise systems

Cons:
- Weaker support for advanced JSON features compared to PostgreSQL
- Fewer built-in analytical capabilities
- Not aligned with the client's specified database requirement

MongoDB
MongoDB is a document-based NoSQL database designed for flexible schemas and horizontal scaling.

Pros:
- Flexible schema makes it easy to iterate quickly during development
- Scales horizontally very well for large distributed systems

Cons:
- Does not enforce relational integrity like traditional SQL databases
- Weaker support for complex transactions
- Not suitable for systems that require strict consistency and structured relationships, such as this one

Final Selection: PostgreSQL

It handles structured relational data very effectively and guarantees transactional integrity, which is essential for features like time tracking and audit logging where accuracy is critical. Its strong indexing and query optimization also make it well-suited for reporting and analytics use cases.

While NoSQL solutions like MongoDB offer flexibility, they trade off consistency and relational integrity, which are important for this system. Overall, PostgreSQL provides the best balance of reliability, performance, and enterprise-level features needed for this application.

Caching Layer

Caching is used to improve performance by temporarily storing frequently accessed data such as API responses, session data, and commonly requested database results.

Technology Selected: Redis
Redis was chosen because it is specifically designed for fast, in-memory data access, which makes it ideal for caching.

In this system, Redis is used to store things like responses from external services (for example Jira and GitHub data) and user session information. This reduces the need to repeatedly call external APIs or query the database, which significantly improves response times and helps the system meet the target of 1–2 second response times.

Compared to relying on a relational database like PostgreSQL for repeated reads, Redis is much faster because it operates entirely in memory. It also supports features like Time-to-Live (TTL), which makes it easy to automatically expire outdated cache entries and keep data fresh without manual cleanup.

API Communication

Technology Selected: RESTful APIs
RESTful APIs are used as the main way for the frontend and backend to communicate. They're a good fit here because they're a widely accepted standard and already align with the client's requirements. Using REST also keeps the frontend and backend clearly separated, which means each part can

be developed, scaled, and maintained independently without tightly depending on the other.

Another benefit is that REST makes it straightforward to integrate external services like Jira, GitHub, and calendar systems, since most modern APIs already follow REST principles. On the backend, Spring Boot makes this even easier by providing strong built-in support for building REST controllers, validating requests and responses, and handling errors in a clean and consistent way. This helps keep the API design structured and aligned with enterprise development practices.

Hosting and Deployment Platform

The application runs on a scalable, secure platform with disaster recovery and high availability.

Technology Selected: AWS (Amazon Web Services)
The application is deployed on AWS, which provides a scalable and secure cloud environment with built-in support for high availability and disaster recovery. AWS was chosen partly because the client already provides access to AWS resources, which makes it the most practical option for the project. This reduces setup effort and allows the system to be built directly on existing infrastructure.

Another major advantage is that AWS supports the system's reliability requirements, including high availability and the ability to recover within a two-hour window in the event of a failure. It also scales easily as usage grows, which is important for a production system.

AWS integrates well with other parts of the architecture too, including CI/CD pipelines, monitoring tools, and managed services like RDS for PostgreSQL and ElastiCache for Redis. This helps keep the infrastructure consistent and easier to manage. While Microsoft Azure was also considered, AWS was selected because it is already available for the project and is widely adopted in enterprise and financial environments.

Authentication & Authorization

The system needs secure, stateless authentication and fine-grained access control for users, roles, and workspaces.

Technology Selected: Spring Security + JWT (JSON Web Tokens)

The system needs a secure way to handle user login, access control, and permissions across different roles and workspaces. To achieve this, we use Spring Security together with JWT. Spring Security takes care of the core authentication and authorization logic. It allows us to define clear role-based access control, such as Admin, Manager, and Developer roles, and ensures users can only access what they are allowed to.

JWT is used to make the system stateless. Instead of storing user sessions on the server, authentication information is encoded into a token that is sent with each request. This makes the system easier to scale because any backend instance can validate the token without relying on shared session storage.

Overall, this approach keeps the system secure, scalable, and flexible, while also leaving room for future integration with standards like OAuth2.

AI and Machine Learning

The system supports predictive analytics, anomaly detection, and AI-driven insights for time tracking and productivity reporting.

Technology Selected: PyTorch + TensorFlow (Python Microservices)

PyTorch and TensorFlow are used to power the AI functionality, which runs in separate Python-based microservices.

This separation is intentional — it keeps the AI workload independent from the main application. That means predictive models and analysis can scale on their own without impacting the performance of the core system.

PyTorch is particularly useful during model development and experimentation, especially for tasks like anomaly detection where models may need frequent tuning and iteration. TensorFlow, on the other hand, is well-suited for deploying models into production environments where stability and performance are more important.

Using Python microservices also makes sense because Python has a very strong ecosystem for machine learning and data science. It allows faster development and gives access to a wider range of AI/ML tools compared to implementing similar functionality directly in Java.

Overall, this approach keeps the AI layer flexible, scalable, and well-suited for continuous improvement while staying decoupled from the main backend system.

External Integrations

The system integrates with Jira, GitHub, and calendar services for time synchronization and data comparison.

Technology Selected: Spring WebClient

Spring WebClient handles integrations with GitHub, Jira, and calendar systems. It supports non-blocking, asynchronous communication, which helps the system handle multiple external API calls without degrading performance.

This approach meets the requirement of comparing tracked time against external data sources (Jira tickets, GitHub issues, calendar events). WebClient's reactive nature ensures efficient resource utilization, which matters especially when dealing with rate-limited external APIs.

CD and Version Control

Building, testing, and deployment happen automatically through a structured workflow.

Technology Selected: Git + GitHub + GitHub Actions

Git and GitHub handle version control, enabling efficient collaboration, code management, and issue tracking. GitHub Projects manages the development process using Scrum methodology, with weekly sprints, regular reviews, and stand-ups to maintain consistent progress.

GitHub Actions implements CI/CD pipelines, automating the build, test, and deployment process. Code pushed to the dev branch deploys to the development environment, while code merged to main triggers production deployment.

This aligns with the requirement for a structured development workflow and ensures consistently delivered working code while reducing human error. Environment-specific configurations are managed through Spring Profiles and GitHub secrets.

Branching strategy

Feature branching was used.

Branch	Purpose	Deployment Target
feature/*	Feature development	Local
dev	Integration branch	Development environment
main	Production-ready code	Production environment

Summary Technology Stack

Layer	Technology
Frontend	Angular 17, Material UI
Backend	Spring Boot 3.5+, Java 17
Database	PostgreSQL
Caching	Redis
Containerization	Docker, Docker Compose
Hosting	AWS (EC2, RDS, ElastiCache, Parameter Store)
Authentication	Spring Security, JWT
AI/ML	PyTorch, TensorFlow (Python microservices)
Testing	JUnit 5, Mockito, Spring Boot Test, Cypress, PyTest
API Communication	RESTful APIs, Swagger / OpenAPI
External Integrations	Spring WebClient
CI/CD	Git, GitHub, GitHub Actions
Monitoring	Prometheus, Grafana
Methodology	Scrum with GitHub Projects

Deployment

Deployment Requirements

The Momently platform adopts a containerised cloud deployment architecture designed to provide portability, scalability, maintainability and deployments. Each subsystem (mainly Frontend, Backend, AI service) is packaged as an independent Docker container, ensuring that every component executes within an isolated and reproducible runtime environment.

The production environment is hosted on a single Amazon EC2 instance where Docker Compose orchestrates the execution of all application services. The application containers are a Spring Boot backend, a FastAPI AI service, a PostgreSQL database and a Redis cache. An Nginx reverse proxy is the public entry point to the application. It routes all incoming client requests to the appropriate internal services.

Continuous Integration (CI) pipeline automatically build and validate each subsystem whenever there are changes being pushed follows, it is the testing and security scanning stages. When all stages has succeeded, the Docker images are published to Amazon Elastic Container Registry (Amazon ECR). The production EC2 instance pulls the latest container images from ECR and recreates the affected containers.

Technologies Used

This deployment architecture combines several technologies each selected to address specific requirements. One of the biggest driver to the selected technology was COST.

As what the client required was to make use of open-source and/or free tier technologies. at the beginning of the project. We decided on having 2 phases: **Phase 1 (Friendly Architecture)** and **Phase 2 (Production Architecture)**

Phase 1 - The objective is to minimize the costs while making use of the cloud deployment.

Phase 2 - Once the system grows, additional AWS and non-AWS services will be used in order to achieve the more non-functional requirements.

Docker

Docker was selected as primary containerisation because it packages application together into portable images. This eliminates the common "works on my machine" problem. This ensures that the identical software executes across all machines.

Docker Compose

Docker Compose is used as the orchestration tool. Docker Compose start all service containers at once using a single docker-compose.prod.yml configuration file. This configuration has all container images, environment variables, health checks, service dependencies, networking and storage. The entire production environment can be recreated using a single command.

Amazon EC2

Amazon EC2 serves as the production host for the application. A virtual machine was selected because it provides complete control over operating system, networking configuration, Docker installation, SSL certificates and deployment process. This provides us flexibility for future scaling and cost effective. Another function of the EC2 instance primarily as a deployment target that executes pre-built Docker images.

Amazon Elastic Container Registry

Amazon ECR acts as the private container registry that stores production Docker images.

After every successful pipeline run, updated images are tagged and pushed to ECR. The EC2 instance retrieves those images during deployment. Using ECR avoids rebuilding software directly on the production server and provides a central repository for versioned application images.

GitHub Actions

GitHub Actions implements the Continuous Integration pipeline. Separate workflows exist for the frontend, backend, and AI service. Each workflow automatically performs several stages including:

- source checkout
- dependency installation
- automated testing
- Docker image construction
- vulnerability scanning using Trivy
- authentication with AWS

- image publication to Amazon ECR

Only successful pipeline executions publish deployable container images, improving deployment reliability.

Nginx

Nginx functions as the application's reverse proxy and HTTPS gateway. Instead of exposing individual application containers on the internet, at risk of direct attacks. All client requests knows only and goes only to Nginx. The functions of Nginx:

- terminates HTTPS using Let's Encrypt SSL certificates
- redirects HTTP traffic to HTTPS
- routes requests to appropriate service:
    - routes frontend requests to Angular application
    - forwards REST API requests to Spring Boot backend
    - forwards AI requests to the FastAPI service
- applies HTTP security headers
- enables GZIP compression
- cache static frontend assets

PostgreSQL

PostgreSQL serves as the primary relational database management system. A Docker volume is attached to the PostgreSQL container to ensure that application data persists even when containers are recreated or updated.

Redis

Redis provides the application's in-memory caching layer. Its primary purpose is to reduce repeated database operations by storing frequently accessed data in memory, improving application responsiveness and reducing database load.

FastAPI

FastAPI hosts the AI subsystem independently from the main backend. Separating AI functionality into its own microservice allows machine learning functionality to evolve independently without affecting the primary Spring Boot application. The service communicates with the backend over Docker's internal bridge network.

Spring Boot

Spring Boot provides the core business logic and REST API for Momently. The backend manages authentication, business rules, database interaction, Redis

integration, and communication with external services while exposing REST endpoints consumed by both the Angular frontend and AI service.

Angular

Angular provides the client-side single-page application (SPA). The compiled production build is served through its dedicated frontend container, while Nginx proxies incoming browser requests to this container. API requests generated by the Angular application are forwarded through Nginx to the backend services.

Architectural decision

Containerisation was selected to improve portability and deployment consistency across environments. By packaging each subsystem independently, updates can be deployed without impacting unrelated services.

A reverse proxy architecture was adopted to simplify client communication. Users interact only with Nginx, while internal services remain inaccessible from the public Internet. This improves security and centralises SSL termination, routing, caching, and HTTP configuration.

Separating the frontend, backend, AI service, database, and cache into independent containers promotes loose coupling and simplifies maintenance. Individual services may be rebuilt, updated, or restarted without requiring the entire application to be redeployed.

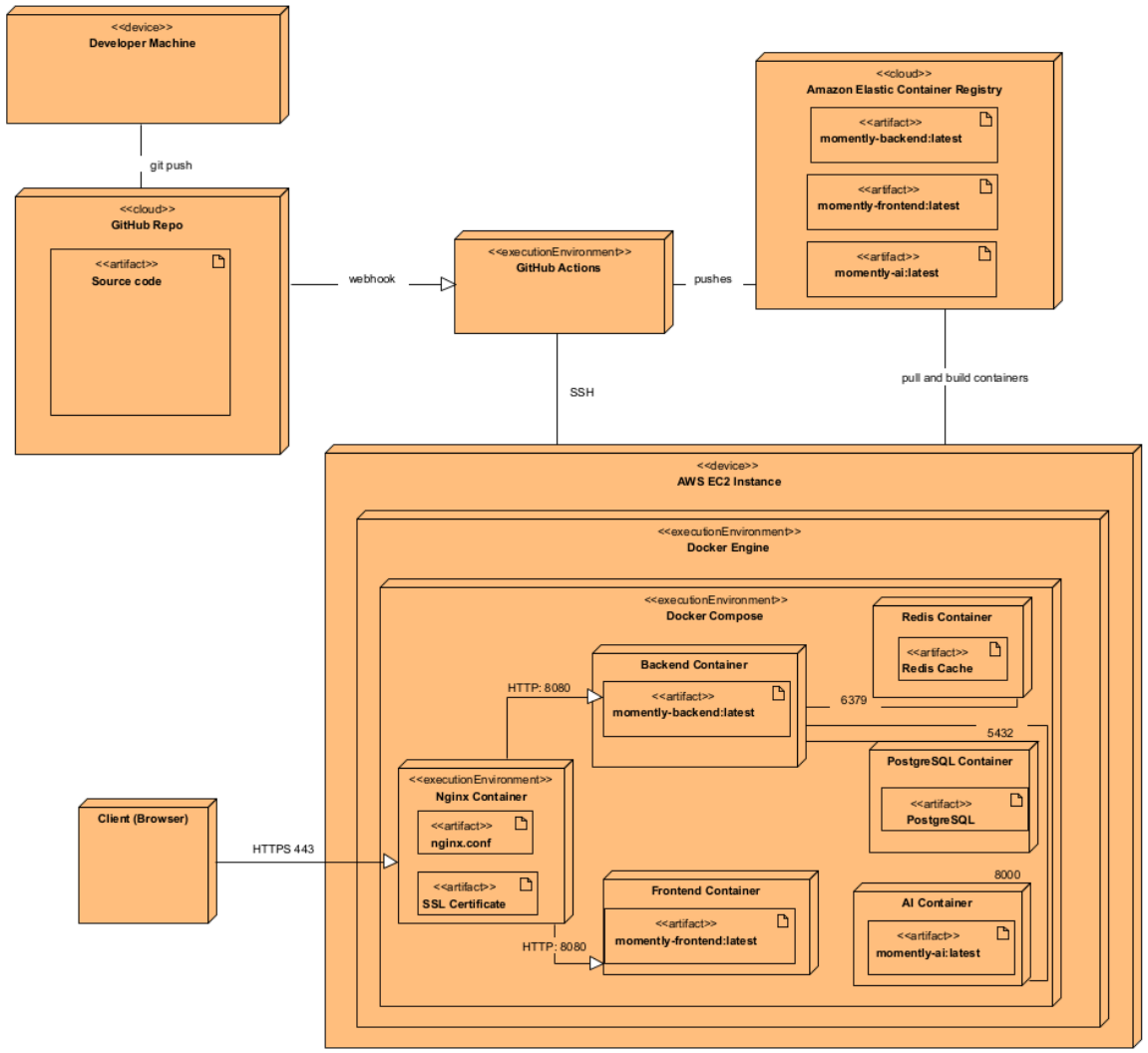
Finally, integrating GitHub Actions with Amazon ECR establishes an automated deployment pipeline in which production images are built, tested, scanned, and published before reaching the production environment. Consequently, the EC2 instance executes only validated Docker images, reducing deployment risk and ensuring consistency across releases.

Artifacts

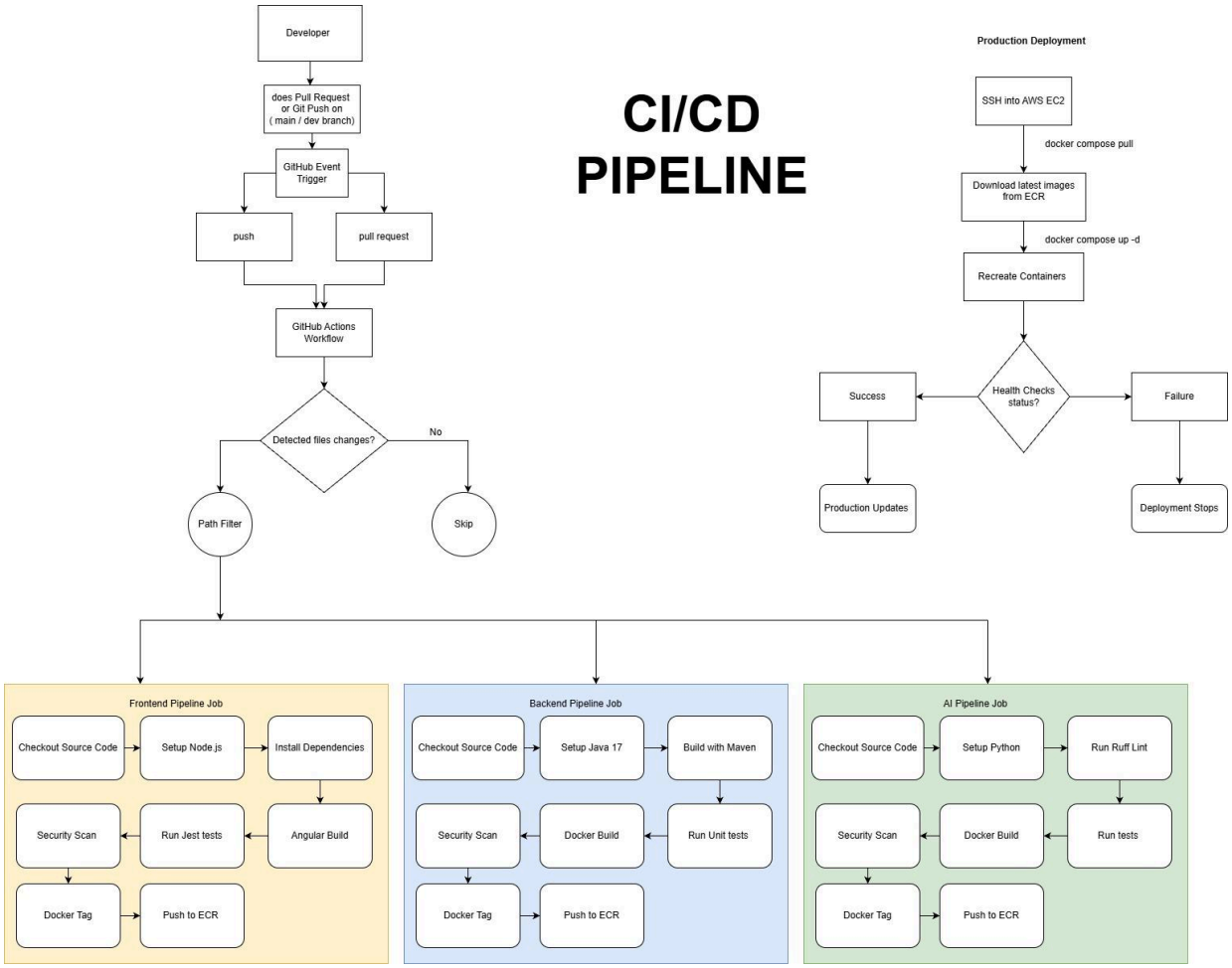
Node	Artifact
Frontend Container	momently-frontend:latest (Angular production bundle)
Backend	momently-backend:latest (Spring Boot JAR)

AI Container	Momently-ai:latest (FastAPI application)
PostgreSQL	Momently database schema + persistent Docker volume
Redis	In-memory cache
Nginx	nginx.conf + SSL certificates
Amazon ECR	Docker image repositories
GitHub Actions	CI/CD workflow definitions

Deployment Diagram



CI/CD Pipeline Diagram



API Contract

Authentication API

Security Notes:

1. ***JWT Tokens***: *the bearer token needs to be included in the authorization header for all the protected endpoints*
2. ***Token expiration***: *the JWT token expires after 24 hours*
3. ***Account locking***: *account is locked for 30 mins, after 5 failed login attempts*

4. ***Email verification***: *users must verify their email before logging in*
5. ***MFA***: *if MFA is enabled, then the user must complete the MFA verification before a JWT token is received*

1. Register New User

Endpoint: POST /api/auth/register

Registers a new user with an email and a password. The user receives a verification email when they register successfully. The email domain must be from the accepted domain of (momentum.co.za or momentum.com). The password would also have to meet the security criteria.

Password requirements:

- *Minimum 8 characters*
- *Atleast one uppercase letter (A-Z)*
- *Atleast one number (0-9)*
- *Atleast one special character (!@#$%^&+=)*
- *No whitespace allowed*

Request Body:

```
{
  "firstName": "string",
  "lastName": "string",
  "email": "string",
  "password": "string"
}
```

Response examples:

SUCCESS (201)

When someone tries to register for the first time.

```
{
  "id": "a5362ed8-3ae0-41a8-b8a4-c97a2ae695b7",
  "email": "thabang.siduke@momentum.co.za",
  "firstName": "Thabang",
  "lastName": "Siduke",
  "createdAt": "2026-07-08T19:21:43.379058942",
  "message": "Verification email sent. Please check your inbox."
}
```

Note: if the user tried to register and they did not verify their email. If they try to register again, they get another verification token.

ERROR (400)
If the password does not meet requirements

```
{
  "message": "password: Password must contain at least one uppercase letter, one number, and one special character (!@#$%^&*)",
  "redirectUrl": null
}
```

If the email already exists.

```
{
  "message": "email already exists",
  "redirectUrl": null
}
```

If the email domain is not accepted.

```
{
  "message": "email: Email must be from the accepted domain",
  "redirectUrl": null
}
```

2. Verify Email

Endpoint: POST /api/auth/verify-email

Verifies a user's email address using the token that they get sent during registration. The token expires after 24 hours. Once they are verified, the user is able to log in.

Request Body example:

```
{
  "token": 8c129fe4-dfa6-4966-8a40-d1a5328958fd
}
```

Response examples:
SUCCESS (200)

```
{
  "message": "Email verified successfully",
  "redirectUrl": "/dashboard"
}
```

ERROR (400)
If the user tries to reuse a token.

```
{
  "message": "token already used",
  "redirectUrl": null
}
```

If they try to use a token that does not exist.

```
{
  "message": "token not found",
  "redirectUrl": null
}
```

If they try to use an expired token.

```
{
  "message": "token expired",
  "redirectUrl": null
}
```

For an edge case: if the user was deleted manually by an admin but their token still exists.

```
{
  "message": "user not found",
  "redirectUrl": null
}
```

3. Login

Endpoint: POST /api/auth/login

Authenticates a user with an email and a password. It validates credentials, checks if the account is locked or verified and returns a JWT token when successfully authenticated.

Request Body:

```
{
  "email": "string",
  "password": "string"
}
```

Response examples:

SUCCESS (200)

When MFA is not enabled


```
{
                                                                    "token":
"eyJhbGciOiJIUzM4NCJ9.eyJzdWIiOiJ0aGFiYW5nLnNpZHVrZUBtb21lbnR1bS5jby
56YSIsInVzZXJJZCI6ImE1MzYyZWQ4LTNhZTAtNDFhOC1iOGE0LWM5N2EyYWU2OTViNy
IsImlhdCI6MTc4MzcyNzUxMCwiZXhwIjoxNzgzODEzOTEwfQ.vjmg8OsFwYEkYjtNRff
X7zEZkRnd1ghy0DZphZX92w9SlA7BtWVnBTNFf79LKbYc",
  "expiresAt": "2026-07-12T01:51:50.750588968",
  "user": {
    "id": "a5362ed8-3ae0-41a8-b8a4-c97a2ae695b7",
    "email": "thabang.siduke@momentum.co.za",
    "firstName": "Thabang",
    "lastName": "Siduke",
    "avatarUrl": null,
    "emailVerified": true,
    "roles": ["ROLE_USER"],
    "mfaEnabled": false
  },
  "requiresMfa": false
}
```


When MFA is enabled, the token is null


```
{
  "token": null,
  "expiresAt": null,
  "user": {
    "id": "a5362ed8-3ae0-41a8-b8a4-c97a2ae695b7",
    "email": "thabang.siduke@momentum.co.za",
    "firstName": "Thabang",
    "lastName": "Siduke",
    "avatarUrl": null,
    "emailVerified": true,
    "roles": ["ROLE_USER"],
    "mfaEnabled": true
  },
  "requiresMfa": true
}
```

ERROR (400)
If they enter the wrong password or email. For safety reasons, they are not informed on which credential they got wrong.

```
{
  "message": "invalid credentials",
  "redirectUrl": null
}
```

If the email is not verified before they try to log in.

```
{
  "message": "please verify your email before logging in",
  "redirectUrl": null
}
```

If they have many failed attempts of trying to log in.

```
{
   "message": "account locked after too many failed attempts. Try again in 30 minutes",
  "redirectUrl": null
}
```

4. Google

Endpoint: POST /api/auth/google

Authenticates the user using Google OAuth2. If the user does not exist, then a new account is created. If the user exists, they are logged in. This supports both new users and existing users linking their Google account.

Request Body:

```
{
  "idToken": "string"
}
```

Response examples:
SUCCESS (200)

```
{
  "token": "string",
  "expiresAt": "2026-07-24T18:16:24.737Z",
  "user": {
```

```
    "id": "string",
    "email": "string",
    "firstName": "string",
    "lastName": "string",
    "avatarUrl": "string",
    "emailVerified": true,
    "roles": [
      "string"
    ],
    "mfaEnabled": true
  },
  "requiresMfa": true
}
```


Projects API

Project status values: ACTIVE, ON_HOLD, COMPLETED, ARCHIVED
Workspace roles (project-level): MANAGER, DEVELOPER

1. Get All Projects

Endpoint: GET /api/projects

Retrieves all the projects based on the user roles. Admins see all projects across all workspaces. Managers see projects in their workspaces. Developers see only the projects that they are assigned to.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "string",
    "description": "string",
```

```
    "status": "string",
    "budgetHours": 0,
    "hourlyRate": 0,
    "budgetCost": 0,
    "startDate": "2026-07-24",
    "endDate": "2026-07-24",
    "myRole": "ADMIN",
    "createdAt": "2026-07-24T18:07:20.145Z",
    "updatedAt": "2026-07-24T18:07:20.145Z"
  }
]
```


2. Get Project Details

Endpoint: GET /api/projects/{projectId}

Retrieves detailed information about a particular project. Users should have access to that specific project. Cost information about budget cost and total cost is only shown to Admin and Managers.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
projectId- string, required

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "string",
  "description": "string",
  "status": "string",
  "budgetHours": 0,
  "hourlyRate": 0,
  "budgetCost": 0,
  "totalCost": 0,
  "members": [
    {
```

```
      "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "firstName": "string",
      "lastName": "string",
      "email": "string",
      "role": "ADMIN",
      "hoursLogged": 0,
      "joinedAt": "2026-07-24T18:08:10.738Z"
    }
  ],
  "hoursLogged": 0,
  "progressPercentage": 0,
  "createdAt": "2026-07-24T18:08:10.738Z",
  "updatedAt": "2026-07-24T18:08:10.738Z"
}
```


3. Create Project

Endpoint: POST /api/projects

Creates a new project in the current workspace. Only Admins and Managers can create projects.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Request Body:


```
{
  "name": "string",
  "description": "string",
  "budgetHours": 0,
  "hourlyRate": 0,
  "budgetCost": 0,
  "startDate": "2026-07-22",
  "endDate": "2026-07-22",
  "managerIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```


Response examples:

SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "string",
  "description": "string",
  "status": "string",
  "budgetHours": 0,
  "hourlyRate": 0,
  "budgetCost": 0,
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "myRole": "ADMIN",
  "createdAt": "2026-07-24T18:14:45.045Z",
  "updatedAt": "2026-07-24T18:14:45.045Z"
}
```


Tasks API

Task status values: TODO, IN_PROGRESS, DONE, BLOCKED
Task priority values: LOW, MEDIUM, HIGH, CRITICAL

Security Notes:

1. ***Admin access:*** *Admins can view and manage all tasks across all projects and workspaces*
2. ***Manager access:*** *Managers can view and manage all the tasks in their workspaces*
3. ***Project Manager access:*** *Project Managers can create tasks in their projects and view all tasks in those projects*
4. ***Developer access:*** *Developers can only view tasks in the projects they are assigned to and either chose to view only their tasks or view all the tasks in that project*
5. ***Task creation:*** *Only Project Managers, Workspace Managers and Admins can create tasks.*
6. ***Parent task:*** *Parent tasks belong to the same project. Preventing orphaned tasks and maintaining project integrity.*
7. ***Soft deletion:*** *Tasks are soft-deleted and remain in the DB for historical reference.*

1. Get Tasks for Project

Endpoint: GET /api/tasks/project/{projectId}

Retrieves all active (non-deleted tasks for a specific project. Users must have access to the project to view the tasks. Admins and Managers can view tasks for any project on their workspace. Developers can view tasks in the projects they are assigned to.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
projectId- string, required

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectName": "string",
    "jiraTicketKey": "string",
    "parentTaskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "string",
    "description": "string",
    "status": "string",
    "priority": "string",
    "estimatedHours": 0,
    "actualHours": 0,
    "assignedToName": "string",
"assignedWorkspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "dueDate": "2026-07-24",
    "completedAt": "2026-07-24T17:34:18.080Z",
    "isDeleted": true,
    "deletedAt": "2026-07-24T17:34:18.080Z",
    "createdAt": "2026-07-24T17:34:18.080Z",
    "updatedAt": "2026-07-24T17:34:18.080Z"
  }
```

]

2. Get My Tasks

Endpoint: GET /api/tasks/my-tasks

Retrieves all (non-deleted) tasks assigned to the user. Personal view showing tasks where the user is the assignee.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectName": "string",
    "jiraTicketKey": "string",
    "parentTaskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "string",
    "description": "string",
    "status": "string",
    "priority": "string",
    "estimatedHours": 0,
    "actualHours": 0,
    "assignedToName": "string",
                                    "assignedWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "dueDate": "2026-07-24",
    "completedAt": "2026-07-24T17:30:45.913Z",
    "isDeleted": true,
    "deletedAt": "2026-07-24T17:30:45.913Z",
    "createdAt": "2026-07-24T17:30:45.913Z",
    "updatedAt": "2026-07-24T17:30:45.913Z"
```

```
  }
]
```

3. Create Task

Endpoint: POST /api/tasks

Creates a new task within that project. Only Admins, Managers (in that workspace), and Project Managers can create tasks. Developers cannot create tasks unless they are assigned as a Project Manager on the Project.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Request Body:
SUCCESS (200)

```
{
  "title": "string",
  "description": "string",
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "jiraTicketKey": "string",
  "parentTaskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "DONE",
  "estimatedHours": 0,
                                        "assignedWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "dueDate": "2026-07-22",
  "priority": "HIGH"
}
```

Response examples:
SUCCESS (200)

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "projectName": "string",
  "jiraTicketKey": "string",
  "parentTaskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "title": "string",
```

```
  "description": "string",
  "status": "string",
  "priority": "string",
  "estimatedHours": 0,
  "actualHours": 0,
  "assignedToName": "string",
                                          "assignedWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "dueDate": "2026-07-24",
  "completedAt": "2026-07-24T17:32:04.375Z",
  "isDeleted": true,
  "deletedAt": "2026-07-24T17:32:04.375Z",
  "createdAt": "2026-07-24T17:32:04.375Z",
  "updatedAt": "2026-07-24T17:32:04.375Z"
}
```


Timesheets API

Timesheet status values: DRAFT, SUBMITTED, APPROVED, REJECTED
Time entry type values: MANUAL, TIMER

Security Notes:

1. ***Developer Access:*** *Developers can only view, submit, and manage their own timesheets and time entries*
2. ***Manager Access:*** *Managers can approve/reject timesheets in their workspace*
3. ***Admin Access:*** *Admins have full access to all timesheets across all workspaces*
4. ***Self-Approval Prevention:*** *Users cannot approve or reject their own timesheets*
5. ***Locking:*** *Once a timesheet is submitted, all time entries are locked and cannot be modified*
6. ***Audit Trail:*** *Approver/Rejector ID and timestamps are recorded for accountability*

1. Get My Timesheets

Endpoint: GET /api/timesheets/me

Retrieves all the timesheets belonging to a user. The users personal view of their timesheet history across all periods.

Headers

Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "periodStart": "2026-07-24",
    "periodEnd": "2026-07-24",
    "status": "string",
    "submittedAt": "2026-07-24T17:39:12.314Z",
    "approvedAt": "2026-07-24T17:39:12.314Z",
                                          "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "rejectedAt": "2026-07-24T17:39:12.314Z",
    "rejectionReason": "string",
    "isLocked": true,
    "lockedAt": "2026-07-24T17:39:12.314Z",
    "createdAt": "2026-07-24T17:39:12.314Z",
    "updatedAt": "2026-07-24T17:39:12.314Z"
  }
]
```


2. Get Timesheets by Status

Endpoint: GET /api/timesheets/me/status/{status}

Retrieves timesheets belonging to the authenticated user filtered by status. Useful for viewing timesheets in a specific state (for example pending approval or still in draft).

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
status - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "periodStart": "2026-07-24",
    "periodEnd": "2026-07-24",
    "status": "string",
    "submittedAt": "2026-07-24T17:40:41.452Z",
    "approvedAt": "2026-07-24T17:40:41.452Z",
                                        "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "rejectedAt": "2026-07-24T17:40:41.452Z",
    "rejectionReason": "string",
    "isLocked": true,
    "lockedAt": "2026-07-24T17:40:41.452Z",
    "createdAt": "2026-07-24T17:40:41.452Z",
    "updatedAt": "2026-07-24T17:40:41.452Z"
  }
]
```


3. Get Timesheet by ID

Endpoint: GET /api/timesheets/{id}

Retrieves a specific timesheet by its ID. Users can only view timesheets they have access to.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "periodStart": "2026-07-24",
  "periodEnd": "2026-07-24",
  "status": "string",
  "submittedAt": "2026-07-24T17:41:18.270Z",
  "approvedAt": "2026-07-24T17:41:18.270Z",
                                        "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "rejectedAt": "2026-07-24T17:41:18.270Z",
  "rejectionReason": "string",
  "isLocked": true,
  "lockedAt": "2026-07-24T17:41:18.270Z",
  "createdAt": "2026-07-24T17:41:18.270Z",
  "updatedAt": "2026-07-24T17:41:18.270Z"
}
```


4. Get Timesheet Entries

Endpoint: GET /api/timesheets/{id}/entries

Retrieves all time entries belonging to a specific timesheet. Provides a detailed breakdown of all time logged within a given period.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "timesheetId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
    "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "startTime": "2026-07-24T17:42:12.444Z",
    "endTime": "2026-07-24T17:42:12.444Z",
    "durationSeconds": 1073741824,
    "entryType": "string",
    "description": "string",
    "isDeleted": true,
    "createdAt": "2026-07-24T17:42:12.448Z",
    "updatedAt": "2026-07-24T17:42:12.448Z"
  }
]
```


5. Submit Timesheet

Endpoint: POST /api/timesheets/{id}/submit

Submits a DRAFT timesheet for approval. This action locks all time entries in the timesheet and changes the status to SUBMITTED. Once submitted, timesheet entries cannot be modified. Only the timesheet owner can perform this action.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "periodStart": "2026-07-24",
  "periodEnd": "2026-07-24",
  "status": "string",
  "submittedAt": "2026-07-24T17:42:56.337Z",
  "approvedAt": "2026-07-24T17:42:56.337Z",
                                    "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
  "rejectedAt": "2026-07-24T17:42:56.337Z",
  "rejectionReason": "string",
  "isLocked": true,
  "lockedAt": "2026-07-24T17:42:56.337Z",
  "createdAt": "2026-07-24T17:42:56.337Z",
  "updatedAt": "2026-07-24T17:42:56.337Z"
}
```


6. Approve Timesheet

Endpoint: POST /api/timesheets/{id}/approve

Approves a SUBMITTED timesheet. This action is performed by a manager or admin to validate the time entries. Once approved, the timesheet is locked and cannot be modified. Users cannot approve their own timesheets.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "periodStart": "2026-07-24",
  "periodEnd": "2026-07-24",
  "status": "string",
  "submittedAt": "2026-07-24T17:45:28.665Z",
  "approvedAt": "2026-07-24T17:45:28.665Z",
                                    "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "rejectedAt": "2026-07-24T17:45:28.665Z",
  "rejectionReason": "string",
  "isLocked": true,
  "lockedAt": "2026-07-24T17:45:28.665Z",
```

```
  "createdAt": "2026-07-24T17:45:28.665Z",
  "updatedAt": "2026-07-24T17:45:28.665Z"
}
```


7. Reject Timesheet

Endpoint: POST /api/timesheets/{id}/reject

Rejects a SUBMITTED timesheet. This action is performed by a manager or admin to indicate the timesheet needs revision. A rejection reason must be provided. The timesheet is unlocked, allowing the user to make corrections and resubmit. Users cannot reject their own timesheets.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:


```
{
  "reason": "string"
}
```


Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "periodStart": "2026-07-24",
  "periodEnd": "2026-07-24",
  "status": "string",
  "submittedAt": "2026-07-24T17:46:07.749Z",
  "approvedAt": "2026-07-24T17:46:07.749Z",
                                        "approvedByWorkspaceMemberId":
"3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "rejectedAt": "2026-07-24T17:46:07.749Z",
  "rejectionReason": "string",
  "isLocked": true,
  "lockedAt": "2026-07-24T17:46:07.749Z",
```

```
  "createdAt": "2026-07-24T17:46:07.749Z",
  "updatedAt": "2026-07-24T17:46:07.749Z"
}
```


Timer API

Timer status values: RUNNING, PAUSED, STOPPED
Time entry types: TIMER, MANUAL

Security Notes:

1. ***One Timer Rule:*** *only one timer can be active at a time across all workspaces for a user. This prevents duplicate time tracking*
2. ***Project Assignment****: users must be assigned to a project to start a timer on it. Unauthorized users cannot track time against projects they don't have access to*
3. ***Task Validation:*** *if a task is provided, it must belong to the specified project. This prevents time entries being associated with incorrect tasks.*
4. ***Timer Persistence:*** *active timers persist across page refreshes and browser restarts. They continue running until explicitly stopped or discarded.*
5. ***Pause/Resume:*** *pause and resume allow users to take breaks without losing time tracking accuracy. Total paused time is subtracted from the final duration.*
6. ***Time Entry Creation:*** *when stopped, a DRAFT time entry is automatically created and associated with the current week's timesheet. This entry can be reviewed and modified before submission.*
7. ***Discard:*** *discarding a timer does NOT create a time entry. This is for accidental timer starts*

1. Start Timer

Endpoint: POST /api/timers/start

Starts a new timer session for the authenticated user. The timer tracks time spent on a specific project and optionally a task. Only one timer can be active at a time across all workspaces for a user. If a timer is already running, a conflict error is returned. If a timer is paused, the user must resume or discard it before starting a new one.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Request Body:

```
{
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Response examples:

```
{
  "id": "c2a07870-124e-48d5-a178-96b40b3ba11c",
  "project": {
    "id": "00000000-0000-0000-0001-000000000040",
    "name": "Mobile App Development"
  },
  "task": {
    "id": "00000000-0000-0000-0002-000000000071",
    "title": "Design Dashboard UI"
  },
  "startedAt": "2026-07-25T14:22:57.978287045",
  "elapsedMinutes": 0,
  "elapsedSeconds": 0,
  "active": true,
  "isPaused": false,
  "pausedAt": null
}
```

2. Pause Timer

Endpoint: POST /api/timers/pause

Pauses the currently running timer. The pause time is recorded so the timer can be resumed later with elapsed time calculation. A timer must be running (not already paused) to be paused. When a user exits the application, their timer remains paused, and can be resumed after they sign in again.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)

```
{
```

```
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "project": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "string"
  },
  "task": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "string"
  },
  "startedAt": "2026-07-24T17:47:19.781Z",
  "elapsedMinutes": 1073741824,
  "elapsedSeconds": 1073741824,
  "active": true,
  "isPaused": true,
  "pausedAt": "2026-07-24T17:47:19.781Z"
}
```


3. Resume Timer

Endpoint: POST /api/timers/resume

Resumes a paused timer. The total paused duration is calculated and accumulated so the final elapsed time accurately reflects only the time the timer was actively running. A timer can be paused and resumed multiple times and even after a user signs out and back in, the timer retains its state and can be resumed.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "project": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "string"
  },
  "task": {
```

```
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "string"
  },
  "startedAt": "2026-07-24T17:49:22.701Z",
  "elapsedMinutes": 1073741824,
  "elapsedSeconds": 1073741824,
  "active": true,
  "isPaused": true,
  "pausedAt": "2026-07-24T17:49:22.701Z"
}
```


4. Stop Timer

Endpoint: POST /api/timers/stop

Stops the currently running timer and creates a DRAFT time entry. The time entry is automatically associated with the current week's timesheet. The total elapsed time is calculated by subtracting any paused durations from the total time.

Headers
Authorization: Bearer {jwt_token}
Request Body: None

Response examples:
SUCCESS (200)


```
{
  "timerId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "stoppedAt": "2026-07-24T17:52:59.911Z",
  "durationSeconds": 1073741824,
  "createdTimeEntry": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "project": {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "name": "string"
    },
    "task": {
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "title": "string"
    },
    "date": "2026-07-24",
```

```
    "startTime": {
      "hour": 1073741824,
      "minute": 1073741824,
      "second": 1073741824,
      "nano": 1073741824
    },
    "endTime": {
      "hour": 1073741824,
      "minute": 1073741824,
      "second": 1073741824,
      "nano": 1073741824
    },
    "durationSeconds": 1073741824,
    "status": "string"
  }
}
```


5. Get Active Timer

Endpoint: GET /api/timers/active

Retrieves the currently running timer for the authenticated user. Returns the timer details including elapsed time, project, task, and pause status. If no timer is active, it returns no content.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "project": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "name": "string"
  },
  "task": {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "title": "string"
```

```
  },
  "startedAt": "2026-07-24T17:55:21.272Z",
  "elapsedMinutes": 1073741824,
  "elapsedSeconds": 1073741824,
  "active": true,
  "isPaused": true,
  "pausedAt": "2026-07-24T17:55:21.272Z"
}
```


6. Discard Timer

Endpoint: DELETE /api/timers/discard

Discards the currently running timer without creating a time entry. This is useful when a user accidentally starts a timer or decides not to track the time. The timer session is permanently deleted.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)

None

Time Entries API

Time entry type values: MANUAL, TIMER
Time entry locking:

1. *DRAFT (not locked, editable/deletable)*
2. *SUBMITTED (locked, uneditable)*
3. *APPROVED (locked, uneditable)*
4. *REJECTED (not locked, editable/deletable)*

Security Notes:

1. ***Owner Only:** users can only view, edit, and delete their own time entries. Admins and Managers cannot modify entries belonging to other users*
2. ***Locking:** time entries belonging to submitted or approved timesheets are locked and cannot be modified or deleted*

3. ***Soft Delete:*** *deleted entries are marked with a flag and remain in the database for audit purposes*
4. ***Timesheet Association:*** *time entries are automatically associated with the correct week's timesheet based on the entry date*
5. ***Timer Integration:*** *entries created from a timer cannot be modified via the manual update endpoint (they must be modified through the timer flow)*
6. ***Validation:*** *all time entries require valid project IDs, start/end times, and positive duration*

1. Create Time Entry (Manual)

Endpoint: POST /api/time-entries

Creates a new manual time entry for the user. The time entry is automatically associated with the current week's timesheet. Users can only create time entries for projects they have access to. Once created, the time entry is in a DRAFT state and can be modified until the timesheet is submitted.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Request Body:

```
{
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "startTime": "2026-07-22T21:40:15.112Z",
  "endTime": "2026-07-22T21:40:15.112Z",
  "durationSeconds": 1073741824,
  "entryType": "string",
  "description": "string"
}
```

Response examples:

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "timesheetId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "startTime": "2026-07-24T18:00:30.993Z",
```

```
  "endTime": "2026-07-24T18:00:30.993Z",
  "durationSeconds": 1073741824,
  "entryType": "string",
  "description": "string",
  "isDeleted": true,
  "createdAt": "2026-07-24T18:00:30.993Z",
  "updatedAt": "2026-07-24T18:00:30.993Z"
}
```


2. Get My Time Entries

Endpoint: GET /api/time-entries/me

Retrieves all time entries belonging to the user, ordered by start time with the most recent first. This is the user's personal view of their time tracking history.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "timesheetId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "startTime": "2026-07-24T18:01:57.371Z",
    "endTime": "2026-07-24T18:01:57.371Z",
    "durationSeconds": 1073741824,
    "entryType": "string",
    "description": "string",
    "isDeleted": true,
    "createdAt": "2026-07-24T18:01:57.371Z",
    "updatedAt": "2026-07-24T18:01:57.371Z"
  }
]
```

3. Get Time Entry by ID

Endpoint: GET /api/time-entries/{id}

Retrieves a specific time entry by its ID. Users can only view their own time entries. This is useful for viewing details of a specific entry before editing or reviewing.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "timesheetId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "startTime": "2026-07-24T18:02:56.232Z",
  "endTime": "2026-07-24T18:02:56.232Z",
  "durationSeconds": 1073741824,
  "entryType": "string",
  "description": "string",
  "isDeleted": true,
  "createdAt": "2026-07-24T18:02:56.232Z",
  "updatedAt": "2026-07-24T18:02:56.232Z"
}
```


4. Update Time Entry

Endpoint: PUT /api/time-entries/{id}

Updates an existing time entry. Users can only update their own time entries. Time entries that are locked (belong to a submitted or approved timesheet) cannot be updated. The update replaces all fields of the time entry.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:


```
{
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "startTime": "2026-07-22T21:42:20.235Z",
  "endTime": "2026-07-22T21:42:20.235Z",
  "durationSeconds": 1073741824,
  "entryType": "string",
  "description": "string"
}
```


Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "timesheetId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "startTime": "2026-07-24T18:03:39.159Z",
  "endTime": "2026-07-24T18:03:39.159Z",
  "durationSeconds": 1073741824,
  "entryType": "string",
  "description": "string",
  "isDeleted": true,
  "createdAt": "2026-07-24T18:03:39.159Z",
  "updatedAt": "2026-07-24T18:03:39.159Z"
}
```

5. Delete Time Entry (soft delete)

Endpoint: DELETE /api/time-entries/{id}

Performs a soft delete of a time entry. The entry is marked as deleted but remains in the database for audit purposes. Users can only delete their own time entries. Time entries that are locked (belong to a submitted or approved timesheet) cannot be deleted.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)
None

Reports API

Security notes:

1. ***Personal Data Only:*** *reports are generated only for the authenticated user. Users cannot view reports for other users*
2. ***Date Range Validation:*** *the 'from' date must be before or equal to the 'to' date to ensure a valid date range*
3. ***Data Privacy:*** *all report data is filtered by the authenticated user's ID, ensuring users only see their own time entries*

1. Get Productivity Report

Endpoint: GET /api/reports/productivity

Generates a productivity report for the authenticated user within a specified date range.Helps users analyze their productivity patterns and time allocation across projects and tasks.

Headers

Authorization: Bearer {jwt_token}

Query Parameters:
from - string(date), required, YYYY-MM-DD
to - string(date), required, YYYY-MM-DD

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "generatedAt": "2026-07-24T18:05:13.847Z",
  "period": {
    "from": "2026-07-24",
    "to": "2026-07-24"
  },
  "summary": {
    "totalHoursLogged": 0.1,
    "totalEntriesLogged": 1073741824,
    "tasksWorkedOn": 1073741824,
    "projectsWorkedOn": 1073741824
  },
  "byTask": [
    {
      "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "taskTitle": "string",
      "projectName": "string",
      "hoursLogged": 0.1,
      "entryCount": 1073741824
    }
  ],
  "byWeek": [
    {
      "week": "string",
      "hours": 0.1,
      "entryCount": 1073741824
    }
  ]
}
```

Insights API

Security Notes:

1. ***Personal Data Only:*** *insights are generated only for the authenticated user. Users cannot view insights for other users*
2. ***Date Range Validation:*** *the 'from' date must be before or equal to the 'to' date to ensure a valid date range*

1. Get Personal Insights Summary

Endpoint: GET /api/insights/summary

Generates a personal insights summary for the authenticated user within a specified date range.

Headers
Authorization: Bearer {jwt_token}

Query Parameters:
from - string(date), required, YYYY-MM-DD
to - string(date), required, YYYY-MM-DD

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "totalHoursLogged": 0.1,
  "averageHoursPerDay": 0.1,
  "totalDaysLogged": 1073741824,
  "hoursPerProject": [
    {
      "projectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "projectName": "string",
      "hours": 0.1,
      "entryCount": 1073741824
    }
  ],
  "hoursPerTask": [
    {
```

```
      "taskId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "taskTitle": "string",
      "hours": 0.1,
      "status": "string"
    }
  ],
  "dailyTrend": [
    {
      "date": "string",
      "hours": 0.1,
      "entryCount": 1073741824
    }
  ]
}
```


Leave-Requests API

Leave Request status values: PENDING, APPROVED, REJECTED, CANCELLED

Leave Request types: ANNUAL, SICK, MATERNITY, PATERNITY, FAMILY_RESPONSIBILITY, OTHER

Security Notes:

1. ***Self-Approval Prevention:** users cannot approve or reject their own leave requests*
2. ***Workspace Isolation:** Managers can only view, approve, and reject requests within their workspace*
3. ***Admin Access:** Admins have full access to all leave requests across all workspaces*
4. ***Cancellation:** only the requester can cancel their own request. Managers/Admins should use REJECT instead.*
5. ***Pending Only:** only PENDING requests can be updated, approved, rejected, or cancelled*
6. ***Overlap Prevention:** users cannot create leave requests that overlap with existing approved leave*

1. Create Leave Request

Endpoint: POST /api/leave-requests

Creates a new leave request for the authenticated user. The request is created with a PENDING status and cannot overlap with existing approved leave requests. The user must be a valid workspace member.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Request Body:

```
{
  "leaveType": "OTHER",
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string"
}
```

Attachments should be in this format:

```
{
    "files": [
        {
        "id": "550e8400-e29b-41d4-a716-446655440000",
        "name": "medical-certificate.pdf",
        "type": "application/pdf",
        "size": 102400,
                                                                    "url":
"https://storage.example.com/uploads/medical-certificate.pdf",
        "uploadedAt": "2026-07-23T10:00:00"
        }
    ]
}
```

Response examples:
SUCCESS (200)

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
  "memberName": "string",
  "leaveType": "string",
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T18:57:00.304Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T18:57:00.304Z",
  "updatedAt": "2026-07-24T18:57:00.304Z"
}
```


2. Update Leave Request

Endpoint: PATCH /api/leave-requests/{id}

Updates an existing pending leave request. Only the requester can update their own requests. Only PENDING requests can be updated.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:
Note: All fields are optional and only provided fields will be updated.


```
{
  "leaveType": "ANNUAL",
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string"
}
```

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "memberName": "string",
  "leaveType": "string",
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T19:05:49.849Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T19:05:49.849Z",
  "updatedAt": "2026-07-24T19:05:49.849Z"
}
```


3. Get My Leave Requests

Endpoint: GET /api/leave-requests/my-requests

Retrieves all leave requests belonging to the authenticated user across all workspaces. Returns requests with the newest first. This is the user's personal view of their leave history.

Headers
Authorization: Bearer {jwt_token}

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
```

```
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "memberName": "string",
    "leaveType": "string",
    "startDate": "2026-07-24",
    "endDate": "2026-07-24",
    "totalDays": 0,
    "reason": "string",
    "attachments": "string",
    "status": "string",
    "approvedByName": "string",
    "approvedAt": "2026-07-24T19:06:25.217Z",
    "rejectionReason": "string",
    "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "createdAt": "2026-07-24T19:06:25.217Z",
    "updatedAt": "2026-07-24T19:06:25.217Z"
  }
]
```


4. Get Leave Request by ID

Endpoint: GET /api/leave-requests/{id}

Retrieves a specific leave request by its ID. Developers can only view their own requests. Managers can view requests in their workspace. Admins can view any request.

Headers
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body: None

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "memberName": "string",
  "leaveType": "string",
```

```
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T19:07:05.343Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T19:07:05.343Z",
  "updatedAt": "2026-07-24T19:07:05.343Z"
}
```


5. Get All Leave Requests (role-based access)

Endpoint: GET /api/leave-requests

Retrieves all leave requests based on the user's role. Developers see only their own requests. Managers see all requests in their workspace. Admins see all requests across all workspaces. Optional status filter applies the same role-based access.

Headers
Authorization: Bearer {jwt_token}

Query Parameters:
status - string, optional
Filters by status (PENDING, APPROVED, REJECTED, CANCELLED)

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "memberName": "string",
    "leaveType": "string",
    "startDate": "2026-07-24",
```

```
    "endDate": "2026-07-24",
    "totalDays": 0,
    "reason": "string",
    "attachments": "string",
    "status": "string",
    "approvedByName": "string",
    "approvedAt": "2026-07-24T19:09:20.470Z",
    "rejectionReason": "string",
    "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "createdAt": "2026-07-24T19:09:20.470Z",
    "updatedAt": "2026-07-24T19:09:20.470Z"
  }
]
```


6. Get Leave Requests by Date Range

Endpoint: GET /api/leave-requests/date-range

Retrieves leave requests within a specified date range based on the user's role. Developers see only their own requests. Managers see all requests in their workspace. Admins see all requests across all workspaces.

Headers
Authorization: Bearer {jwt_token}

Query Parameters:
from - string(date), required, YYYY-MM-DD
to - string(date), required, YYYY-MM-DD

Request Body: None

Response examples:
SUCCESS (200)


```
[
  {
    "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "memberName": "string",
    "leaveType": "string",
    "startDate": "2026-07-24",
```

```
    "endDate": "2026-07-24",
    "totalDays": 0,
    "reason": "string",
    "attachments": "string",
    "status": "string",
    "approvedByName": "string",
    "approvedAt": "2026-07-24T19:07:55.526Z",
    "rejectionReason": "string",
    "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "createdAt": "2026-07-24T19:07:55.526Z",
    "updatedAt": "2026-07-24T19:07:55.526Z"
  }
]
```


7. Approve Leave Request

Endpoint: POST /api/leave-requests/{id}/approve

Approves a pending leave request. Only Admins and Managers can approve requests. Managers can only approve requests in their workspace. Users cannot approve their own requests.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:
{}

Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "memberName": "string",
  "leaveType": "string",
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
```

```
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T19:10:06.351Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T19:10:06.351Z",
  "updatedAt": "2026-07-24T19:10:06.351Z"
}
```


8. Reject Leave Request

Endpoint: POST /api/leave-requests/{id}/reject

Rejects a pending leave request with a reason. Only Admins and Managers can reject requests. Managers can only reject requests in their workspace. Users cannot reject their own requests.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:


```
{
  "reason": "string"
}
```


Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "memberName": "string",
  "leaveType": "string",
  "startDate": "2026-07-24",
```

```
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T19:10:42.692Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T19:10:42.692Z",
  "updatedAt": "2026-07-24T19:10:42.692Z"
}
```


9. Cancel Leave Request

Endpoint: POST /api/leave-requests/{id}/cancel

Cancels a pending leave request. Only the requester can cancel their own request. Managers and Admins cannot cancel someone else's request; they should use REJECT instead.

Headers
Content-Type: application/json
Authorization: Bearer {jwt_token}

Path Parameter:
id - string, required

Request Body:


```
{
  "reason": "string"
}
```


Response examples:
SUCCESS (200)


```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "workspaceMemberId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "memberName": "string",
  "leaveType": "string",
```

```
  "startDate": "2026-07-24",
  "endDate": "2026-07-24",
  "totalDays": 0,
  "reason": "string",
  "attachments": "string",
  "status": "string",
  "approvedByName": "string",
  "approvedAt": "2026-07-24T19:11:41.762Z",
  "rejectionReason": "string",
  "availabilityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "createdAt": "2026-07-24T19:11:41.762Z",
  "updatedAt": "2026-07-24T19:11:41.762Z"
}
```


Database Architecture and Data Model

Database Overview

The system uses PostgreSQL as its primary relational database. PostgreSQL was chosen because it provides reliable relational data modelling, supports complex queries and handles concurrent users effectively.

These features make it well suited for Momently because the database needs to keep time entries and timesheets accurate and consistent, while also supporting reporting, approval workflows, external integrations, and AI-generated insights.

The database is organised into the following main domains:

- **Identity and Authentication** - Users, authentication, MFA, SSO providers, email verification, and password reset functionality.
- **Workspaces and Access Control** - Workspaces, workspace membership, user roles, and workspace-level configuration.
- **Projects and Tasks** - Projects, project membership, tasks, task status, and project budgets.
- **Time Tracking and Timesheets** - Timer sessions, time entries, timesheets, and timesheet approval workflows.

- **Leave and Availability** - Leave requests and workspace member availability.
- **External Integrations** - Synchronised data from Jira, GitHub, and Google Calendar.
- **AI and Analytics** - AI-generated insights, productivity analysis, anomaly detection, and forecasting.
- **Audit and Notifications** - Audit logging and system notifications.

The detailed structure of each database table, including its columns, data types, and descriptions, is provided in Appendix A: Database Table Definitions.

Database Design Decisions

UUID Primary Keys

All major entities use UUIDs as their primary keys and are automatically generated using a PostgreSQL UUID random generator function. They were chosen because:

1. UUIDs are unique not just within our database, but everywhere. This means if we ever need to merge data from different environments or sync with external systems, we won't run into ID clashes.
2. Avoid exposing sequential database identifiers. UUIDs are random and unpredictable, making it much harder for anyone to accidentally or maliciously access data they shouldn't see.
3. UUIDs are designed to work across multiple servers and databases without coordination. This makes it easier to scale the system horizontally, move data between regions, or even switch database providers later on.
4. Backend can handle UUIDs natively. Java has a built-in UUID type.

Workspace-based Multi-Tenancy

The workspaces table serves as the root tenant entity. All user-related activity (projects, tasks, time entries, and timesheets) is contained within a workspace context. A user can belong to multiple workspaces, but data is strictly isolated between them.

This structure provides:

1. **Data isolation** - users can only see data belonging to workspaces they are members of
2. **Flexible membership** - users can belong to multiple workspaces and switch between them
3. **Clear ownership** - every piece of data can be traced back to a workspace and a specific user

Role-based Access control

Access control is through the workspace_members table, which links users to workspaces with a specific role:

1. Admins have full system access across all workspaces and can manage users, projects, settings, and all data.
2. Managers can manage team members, approve timesheets and leave requests, and oversee project work.
3. Developers have standard user permissions for time tracking, viewing assigned projects, and managing personal data.

This design supports the system's RBAC requirements without duplicating user data and allows flexible permission management at the workspace level.

Soft Deletion and Historical Data

Several core tables (projects, tasks, time_entries) implement soft deletion using is_deleted and deleted_at columns.

This approach:

- Preserves historical data for auditing and reporting purposes
- Allows users to hide or remove records without permanent data loss
- Ensures that historical reports remain accurate even after items are deleted

External Integration Architecture

External services have specific integration tables instead of storing integration data directly in core business tables. This supports the addition of future integrations without requiring major changes to the core database model.

AI and Analytics Data

The audit_logs table records significant system actions and changes. This supports accountability and allows important actions such as the following to be traced. Audit records may include:

- The workspace where the action occurred
- The workspace member who performed the action
- The affected project
- The action performed
- Previous values
- New values
- IP addresses
- Timestamps

Authentication and Identity

Authentication-related data is separated into dedicated tables. This separation allows the system to support multiple authentication mechanisms, and extensibility when more mechanisms are added.

References

1. GeeksforGeeks (2026b) Spring Boot Architecture. https://www.geeksforgeeks.org/springboot/spring-boot-architecture/.

Appendices

Appendix A: Database Table Definitions

(pending- to be added)